HOME WORK-1

Name: Hemanjali Gasada NetId: gasada

1. Implementation & Design Decisions

1.1 Adaptive Resizing (Scale)

- **Aspect Ratio Preservation:** When size is specified as a single scalar integer, the algorithm checks the image dimensions to determine the shorter edge. It scales the shorter edge to size while setting the longer edge proportionally using:

$$new_side = round\left(size \times \frac{long_side}{short_side}\right)$$

  This avoids spatial distortion of object geometry.

- **Integer Dimension Rounding:** round() is explicitly applied before casting to int to prevent down-truncation errors that cause single-pixel aspect ratio drift during repeated rescalings.

- **Stochastic Interpolation Selection:** During data augmentation, interpolation methods are sampled randomly from self.interpolations (e.g., cv2.INTER_LINEAR, cv2.INTER_CUBIC, cv2.INTER_NEAREST). This introduces subtle high-frequency variation and blur artifacts that increase model robustness to varying image quality.
-

1.2 Color Jittering (RandomColor)

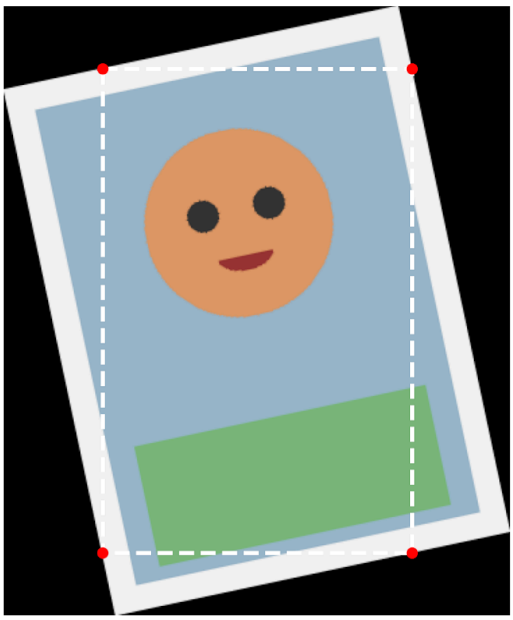
- **Type Conversion & Overflow Prevention:** Direct channel multiplication on standard 8-bit image arrays (uint8) causes integer overflow/wrap-around modulo arithmetic (e.g., 200 * 1.4 = 280→ 24). To prevent color corruption, pixel values are converted to 32-bit floats during scaling, clipped to $[0, 255]$, and explicitly cast back to uint8.

- **Channel-Wise Independence:** A distinct multiplier $\alpha_c \sim U(1 - r, 1 + r)$ is drawn independently for each color channel $c \in \{0, 1, 2\}$, allowing the network to become invariant to white-balance shifts and tint variations.
- **Look-Up Table (LUT) Acceleration:** Rather than performing $H \times W \times C$ floating-point multiplications for every pixel in Python, a 256-element array is precomputed per channel and applied using OpenCV's cv2.LUT(). This reduces runtime complexity from $O(H \cdot W \cdot C)$ to $O(256 \cdot C)$, eliminating CPU bottlenecking in the data loader.

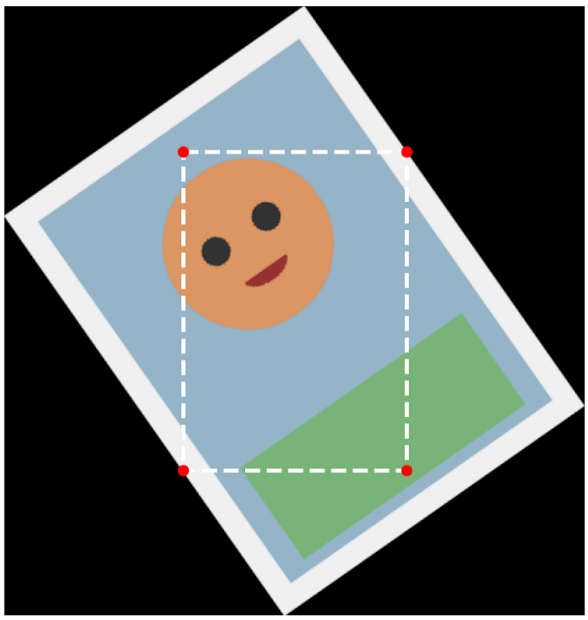
1.3 Random 2D Rotation & Max-Area Cropping (RandomRotate)

Pictorial representation to understand 4 side touch and 2 side touch :

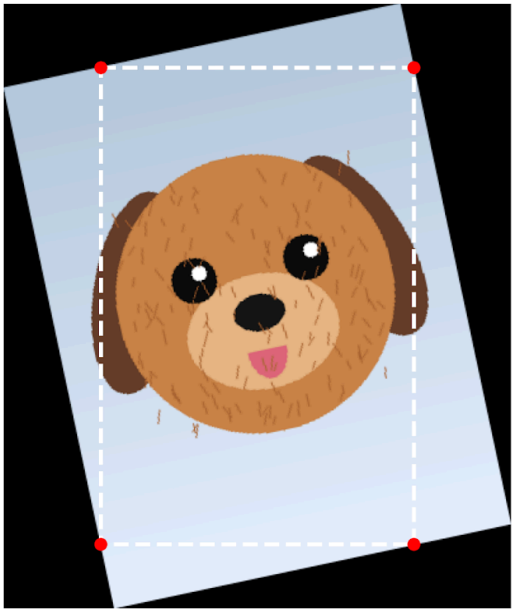
4-Side Touch
Rotation Angle = 12°



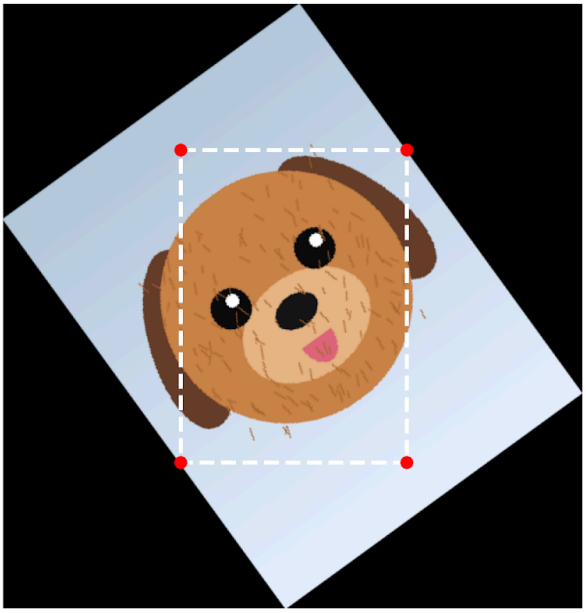
2-Side Touch
Rotation Angle = 35°



Case 2: 4-Side Touch
(Small Angle: 12°)

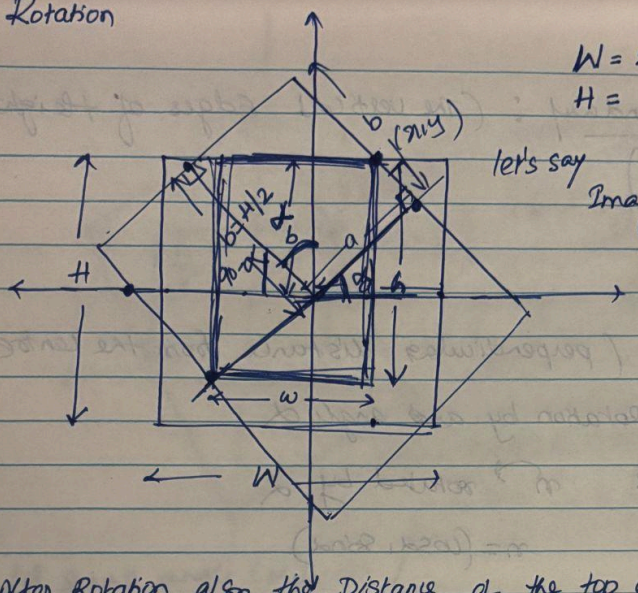


Case 1: 2-Side Touch
(Large Angle: 36°)



Derivation :

3.2 Rotation



⇒ After Rotation also the Distance of the top edge remains fixed at perpendicular distance $d = H/2$ from the centre.

$$\hat{n} = (-\sin\alpha, \cos\alpha) = (-\cos(90-\alpha), \cos\alpha)$$

normal vector

$$= (-\sin\alpha, \cos\alpha)$$

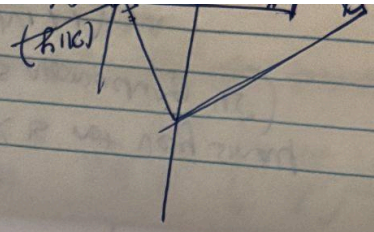
unit normal vector $n_0 = (0,1)$

→ Taking the absolute positive projection of 1st quadrant corner $(x,y) = \left(\frac{w}{2}, \frac{h}{2}\right)$:

$$n = (\sin\alpha, \cos\alpha)$$

$$n \cdot (x,y) = d \Rightarrow x\sin\alpha + y\cos\alpha = \frac{H}{2}$$

→ (w,h) → new width, height of the max rectangle possible.



⇒ Side W Boundary : (The vertical edges of height H)

Before ($\alpha = 0°$)

$x = \frac{W}{2}$

$n_0 = (1, 0)$

$d = W/2$ (perpendicular distance from the centre (0,0))

↳ After Rotation by and angle α

$d = W/2$ n ↷ rotated by α

$n_{initial} = (1, 0)$ $n = (\cos\alpha, \sin\alpha)$

$n \cdot (x, y) = d \Rightarrow x\cos\alpha + y\sin\alpha = \frac{W}{2}$

⇒ Inner Rectangle : centered at (0,0)

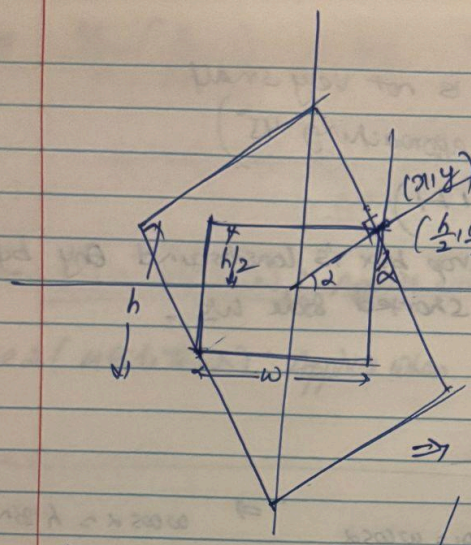
Corner Corners : the top-right corner of the crop box located at $(x, y) = \left(\frac{w}{2}, \frac{h}{2}\right)$

⇒ Boundary constraints : The outer rotated frame forms two pairs of parallel lines tilted by angle α

⇒ Side W Boundary : Normal vector is $(\cos\alpha, \sin\alpha)$

↳ distance from origin $= W/2$

Side H Boundary : Normal vector is $(\sin\alpha, \cos\alpha)$ ⟶ distance from origin $= H/2$



$x\cos\alpha + y\sin\alpha \le \frac{W}{2}$

$x\sin\alpha + y\cos\alpha \le \frac{H}{2}$

$\left(x,y = \frac{w}{2}, \frac{h}{2}\right)$

$\Rightarrow w\cos\alpha + h\sin\alpha \le W \rightarrow (1)$

$w\sin\alpha + h\cos\alpha \le H \rightarrow (2)$

$\Rightarrow$ for small angle (α) the crop box touches 4 corners

Solving for w_{max}.

A $\leftarrow$ Eq^n (1) $\times \cos\alpha = w\cos^2\alpha + h\sin\alpha\cos\alpha = W\cos\alpha$

B $\leftarrow$ Eq^n (2) $\times \sin\alpha = w\sin^2\alpha + h\sin\alpha\cos\alpha = H\sin\alpha$

(A) − (B) $= w(\cos^2\alpha - \sin^2\alpha) = W\cos\alpha - H\sin\alpha$

$w(\cos(2\alpha)) = W\cos\alpha - H\sin\alpha$

$$w_{crop} = \frac{W\cos\alpha - H\sin\alpha}{\cos(2\alpha)}$$

Similarly solving for h_{crop}

$$h_{crop} = \frac{H\cos\alpha - W\sin\alpha}{\cos(2\alpha)}$$

Case 1: 2 Side Touch ⇒ α is not very small (α is approaching 45°)

↳ $W_s = \min(W, H)$

↳ Here the drop box is constrained only by the shorted side W_s.

$w\cos\alpha + h\sin\alpha = W_s$

$A = wh \rightarrow$ To maximize it

↳ By AM GM Inequality $A_1 = w\cos\alpha$ ⇒ $w\cos\alpha = h\sin\alpha$

$A_2 = h\sin\alpha$

$\frac{A_1+A_2}{2} = \sqrt{A_1 A_2}$ when $A_1 = A_2$ $\quad W_s = 2w\cos\alpha$

$\boxed{h = \frac{W_s}{2\sin\alpha}}$ $\quad h = \frac{W_s \times \cos\alpha}{2\cos\alpha\sin\alpha}$ $\quad \boxed{w = \frac{W_s}{2\cos\alpha}}$

$\boxed{A_{max} = wh = \frac{W_s^2}{4\sin\alpha\cos\alpha} = \frac{(\min[W,H])^2}{2\sin(2\alpha)}}$

Transition Condition:

$W_l = \max(W, H)$

Case 2: 4 Side touch:

~~$w\cos\alpha + h\sin\alpha \le W_l$~~

$w\sin\alpha + h\cos\alpha \le W_l$

$\boxed{\frac{W_s}{2\cos\alpha}\sin\alpha + \frac{W_s}{2\sin\alpha}\cos\alpha \le W_l}$

Case 1: 2 side Touch ⇒ α is not very small

$$\Rightarrow \frac{w_s}{2}\left(\frac{2}{\sin 2\alpha}\right) \leq w_e \Rightarrow \frac{w_s}{\sin(2\alpha)} \leq w_e$$

$$\sin(2\alpha) \geq \frac{w_s}{w_e} = \frac{\min(W,H)}{\max(W,H)}$$

Case 1 (2 sideTouch): Applies when $\sin(2\alpha) \geq \frac{\min(W,H)}{\max(W,H)}$

Case 2 (4 SideTouch): Applies when $\sin(2\alpha) < \frac{\min(W,H)}{\max(W,H)}$

- **Identity Early-Exit Optimization:** If a sampled angle θ satisfies |θ |< 1.0 degree matrix transformation and geometry calculations are skipped, returning the original image directly. This avoids unnecessary interpolation blurs on near-zero rotations.

- **Exact Analytic Interior Crop:** Applying cv2.warpAffine() introduces black zero-padded corners. To eliminate these without arbitrary manual crops, the maximum-area valid interior rectangle (W_{crop}, H_{crop}) is determined analytically based on α = |θ |mod π/2

    - $Condition: \sin(2\alpha) \geq \frac{\min(W,H)}{\max(W,H)}$
        - **2-Side Touch Case:** If the condition holds true, the inner rectangle touches 2 sides of the bounding region:
        - $W_{crop} = \frac{\min(W,H)}{\sin\alpha}, H_{crop} = \frac{\min(W,H)}{\cos\alpha}$
        - **4-Side Touch Case:** Otherwise, it touches all 4 sides:
        - $W_{crop} = \frac{W\cos\alpha - H\sin\alpha}{\cos(2\alpha)}, H_{crop} = \frac{H\cos\alpha - W\sin\alpha}{\cos(2\alpha)}$
- **Centered Slice Extraction:** The bounding box is sliced from the exact center of the transformed image $(c_x, c_y) = (W/2, H/2)$ using integer slicing $[y_1 : y_2, x_1 : x_2]$ to guarantee spatial symmetry.

1.4 Sequential Pipeline & Structural Ordering (Compose)

- **Pipeline Modular Design:** Compose operates by iteratively passing the output NumPy array of transform i as the input to transform $i + 1$.

- **Optimal Processing Sequence:** Transforms are ordered intentionally as:

- RandomSizedCrop → RandomHorizontalFlip → RandomRotate → RandomColor → ToTensor
    1. Spatial crops are executed early to lower total image resolution before computationally expensive color jittering and rotation matrix warps.
    2. RandomRotate precedes final formatting to ensure all zero-padded borders are cropped before image normalization.
    3.
    4. ToTensor() is placed last to convert NumPy uint8 arrays into PyTorch floating-point Tensors normalized to $[0, 1]$ for model training.

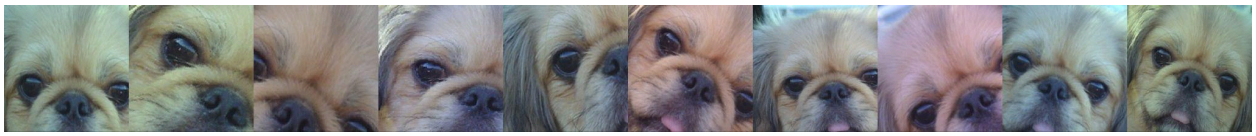
Results & Discussion:

Below are the images of Original image and the Augmented images(After applying all the Transformations) of Dog,Bird, Cat,Plane

Original Image of Dog:



Augmented images of Dog:



Original Image of Bird:



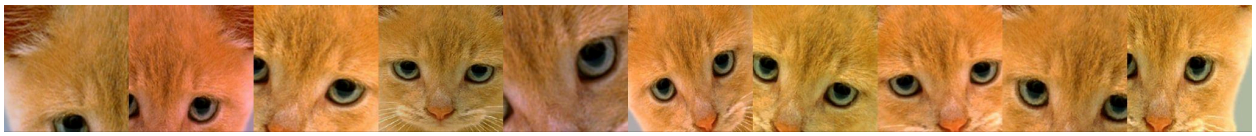
Augmented Images of Bird:



Original Image of Cat:



Augmented Images of Cat:



Original Image of Plane:



Augmented Images of Planes:



From the visual results generated across the test samples (Dog, Bird, Cat, and Plane), we observe that the augmented images differ significantly from the original images in **geometric pose, framing composition, and photometric properties**. Spatially, `RandomRotate` and `RandomHorizontalFlip` alter object orientation and mirror direction. Compositionally, `RandomSizedCrop` alters the target aspect ratio and scale, isolating localized sub-regions (e.g., facial features) versus full-body perspectives. Photometrically, `RandomColor` introduces channel-wise multiplicative perturbations that simulate varying ambient lighting and sensor white-balance conditions. Crucially, while low-level pixel values and spatial alignments change dramatically across iterations, the high-level semantic class identity remains fully preserved

Semantic Label Preservation: Despite significant spatial cropping, rotation, flipping, and color distortion, all augmented samples strictly preserve their ground-truth class identity (e.g., a jet remains recognizable as a plane).
Spatial Augmentations force the model to learn translation-, scale-, and rotation-invariant feature descriptors.

Photometric Augmentations decouple class predictions from lighting conditions, shadow intensity, and sensor white-balance variations.

Answers for Questions ins 3.3:
Q1: What is the benefit of PyTorch dataloader? Please compare the dataloader to a solution that simply loops over a file list and batches the images.

The main benefit of PyTorch dataloader is parallelism. It is optimised to take advantage of the huge number of cores of the GPU and run the operations in parallel. It also supports pipeline parallelism like parallelising data transfer and matrix computations.
Two of the Key Differences and Benefits :

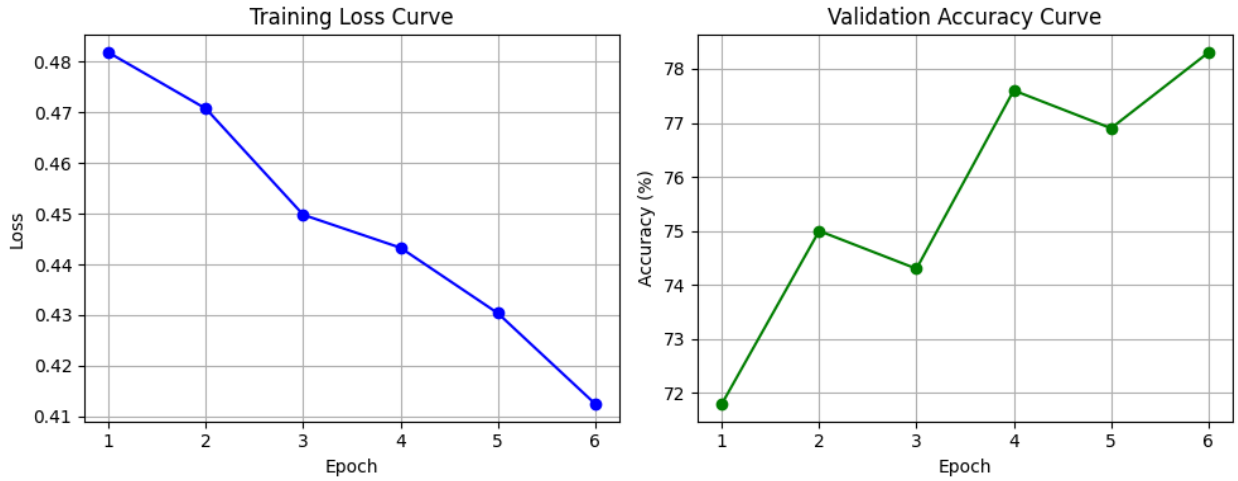
Asynchronous Multiprocessing :

- **Naive File Loop:** Runs synchronously on the main Python thread. The CPU reads an image from disk, decodes it and applies augmentations (flipping, cropping, color jittering)(involves math computations). While the CPU executes these steps, the GPU sits idle. Once the batch reaches the GPU, the CPU sits idle waiting for the GPU to finish forward/backward passes.
- **PyTorch DataLoader:** Spawns multiple background subprocesses (`num_workers > 0`). These worker processes continuously load, decode, and transform future batches in the background while the GPU trains on the current batch. This pipeline parallelism completely hides I/O and CPU transformation latency, ensuring near 100% GPU utilization.

Memory Management and Host-to-Device Transfer (`pin_memory=True`)

- **Naive File Loop:** Loads data into standard pageable system memory. When transferring data from host CPU memory to GPU VRAM via PCIe, the OS must first copy the data to a temporary "pinned" (page-locked) host buffer, creating a double-copy overhead.
- **PyTorch DataLoader:** Setting `pin_memory=True` allocates tensors directly in page-locked CPU memory. This enables **Direct Memory Access (DMA)**, allowing fast, asynchronous CPU-to-GPU memory transfers (`non_blocking=True`) that bypass CPU involvement entirely during transfer.

Q2:Please briefly describe the testing accuracy and training loss curve



From the curves we can see that the training loss decreases across all 6 epochs. It starts at approximately 0.482 in Epoch 1 and decreases steadily to 0.412 by Epoch 6.This smooth, uninterrupted drop confirms that the AdamW optimizer is successfully updating the network parameters and minimizing cross-entropy loss without gradient instability or divergence.
The validation accuracy displays an overall **upward trend**, rising from **71.8%** in Epoch 1 to a peak of **78.3%** in Epoch 6. The minor intermediate fluctuations are typical characteristics of Stochastic Gradient Descent (SGD) paired with randomized data augmentations.

Q3 Answer:
The order of image transformations in a data augmentation pipeline matters significantly and cannot be arbitrarily switched due to both geometric non-commutativity and data-type constraints.
Mathematically, spatial matrix transformations are non-commutative ($A \cdot B \neq B \cdot A$). For instance, applying RandomRotate before RandomSizedCrop introduces black padded boundary corners that may subsequently get cropped into the final image, introducing artificial border noise. Conversely, cropping prior to rotating ensures all sampled pixels represent real scene content.

Operationally, execution order is constrained by data structure representations: transforms relying on OpenCV/NumPy operate on uint8 NumPy arrays in [H, W, C] format. Placing ToTensor() earlier in the pipeline converts data into [C, H, W] PyTorch Tensors, breaking downstream NumPy-based transformations. Therefore, deterministic pipeline ordering is essential for both mathematical validity and program execution.

Question4:

The cross-entropy loss is

$$-\sum_{j=1}^{C} y_j \log p(C_j|\boldsymbol{x}).$$

Because the target vector y is one-hot encoded, $y_k = 1$ for the correct class k and $y_j = 0$ for all incorrect classes $j \neq k$. The summation simplifies to:

$$\mathcal{L} = -\sum_{j=1}^{C} y_j \log p(C_j \mid x) = -\log p(C_k \mid x)$$

Minimizing this loss L means maximizing p(C_k | x) towards 1.0.

This formulation makes complete sense for classification due to the probability coupling introduced by the Softmax function.

Although non-target classes $j \neq k$ have target values $y_j = 0$, they are not ignored. Expanding p(C_k | x) reveals the loss function as $L = -o_k + \log\left(\sum_{c=1}^{C} e^{o_c}\right)$. To minimize this quantity, gradient descent must simultaneously maximize the target logit o_k while suppressing all non-target logits o_j present in the log-sum-exp denominator. Furthermore, because Softmax constrains output probabilities to sum to 1.0, driving p(C_k | x)to 1.0 inherently forces the sum of all incorrect class probabilities to zero.

AI Disclusure:

I used Gemini to understand the code, functions and also asked it to explain the concepts.